



Bansilal Ramnath Agarwal Charitable Trust's Vishwakarma
Institute of Information Technology

**Department of
Artificial Intelligence and Data Science**

Student Name: Sathe Onkar Abasaheb

Class: TY

Division: B

Roll No: 372078

Semester: V

Academic Year: 2025-2026

Subject Name & Code: Cyber Security (AD31234B)

Assignment No: 02

Title of Assignment:

Compare and contrast substitution and transposition techniques and implement any two Methods.
(Caesar cipher, Playfair cipher, Hill cipher, Vigenere cipher, Rail fence - row and column transformation)

Date of Performance: 21/07/2025

Date of Submission: /08/2025

Assignment 2: Compare and contrast substitution and transposition techniques and implement any two Methods. (Caesar cipher, Playfair cipher, Hill cipher, Vigenère cipher, Rail fence - row and column transformation)

Objective:

The objective of this assignment is to compare and contrast substitution and transposition techniques and implement any two methods.

Materials Required:

Computer with Python programming environment.

Introduction to Cryptography

Cryptography, derived from the Greek words *kryptos* (hidden) and *graphein* (to write), is the science of **secure communication** in the presence of adversaries. The two most fundamental categories of classical cryptographic techniques are:

- **Substitution Techniques** – Where symbols are replaced.
- **Transposition Techniques** – Where symbols are repositioned.

These methods were widely used in **ancient and pre-digital cryptographic systems** and form the building blocks of modern encryption strategies.

Substitution Technique:

Definition:

In substitution, each element (character or group of characters) in the plaintext is **systematically replaced** with another symbol according to a fixed system.

Core Idea:

- The **order of characters is maintained**.
- Each character is transformed into another, based on an algorithm or key.
- Can be **monoalphabetic** (single substitution per letter) or **polyalphabetic** (changing substitution based on position or keyword).

Historical Use:

- Julius Caesar used a form of substitution cipher (now named after him) to send military commands.

- The **Vigenere Cipher**, once considered unbreakable, was used for centuries in diplomatic communications.

Advantages:

- **Easy to implement and understand**, especially for educational purposes.
- With complex keys (like in Vigenere), **resistance to simple attacks increases**.

Limitations:

- Most substitution ciphers are **vulnerable to frequency analysis**, especially monoalphabetic ones.
- Patterns in the plaintext often persist in the cipher text.

Transposition Technique: Concept and Theory

Definition:

In transposition, characters in the plaintext are **rearranged** based on a specific rule or pattern. The original characters are preserved but their **positions are changed**.

Core Idea:

- Does **not replace** characters.
- Alters the **sequence or arrangement** using keys or patterns.
- Decryption involves reconstructing the original order.

Historical Use:

- The **Scytale cipher**, used by the Spartans, is a cylindrical transposition method.
- Rail Fence and Columnar Transposition were used during **World Wars I and II**.

Advantages:

- Preserves letter frequencies, which makes **frequency analysis less useful**.
- Can be significantly stronger when **combined with substitution**.

Limitations:

- Still vulnerable to pattern-based and brute-force attacks.
- Key management (like number of rails or column order) becomes **essential**.

Implementation of Selected Methods

Caesar Cipher – A Classical Substitution Cipher

Working Principle:

- Each letter is shifted a fixed number of places along the alphabet.
- For example, with a shift of 3: $A \rightarrow D$, $B \rightarrow E$, $Z \rightarrow C$, etc.
- Decryption involves shifting in the **opposite direction**.

Encryption Formula:

Encrypted Letter = (Original Letter + Shift) mod 26

Decryption Formula:

Decrypted Letter = (Encrypted Letter - Shift) mod 26

Security Insight:

- Only 25 possible keys for the English alphabet (excluding 0), making it **easily brute-forceable**.
- Any intercepted cipher text can be cracked using a **frequency analysis** or exhaustive key trial.

Rail Fence Cipher – A Row Transposition Cipher

Working Principle:

- Characters are written in a **zigzag fashion** across a set number of rails.
- The cipher text is obtained by **reading row by row**.
- Decryption is done by **reconstructing the rail pattern** using the cipher length and number of rails.

Example (3 Rails):

Plaintext: **HELLOCRYPTO**

Zigzag Writing:

H	L	C	P		
E	L	O	R	Y	T
L	O	O			

Cipher text (row-wise): **HLCPELORYTLOO**

Security Insight:

- Harder to break than Caesar due to **position scrambling**.
- However, if the rail count is known, decryption becomes straightforward.

- Best used as **part of a compound cipher**.

In-depth Comparison: Substitution vs Transposition

Criteria	Caesar Cipher (Substitution)	Rail Fence Cipher (Transposition)
Technique Type	Substitution	Transposition
Operation	Changes actual letters	Changes position/order of letters
Complexity	Very Simple	Moderate
Security Level	Very Weak (easy to break)	Moderate (if key is hidden)
Frequency Pattern	Not preserved	Preserved
Brute Force Risk	High (only 25 keys)	Lower (depends on rails and length)
Decryption Requirement	Simple shift reversal	Exact rail count and zigzag pattern reconstruction
Use in Real Systems	No (educational use only)	No (used historically, rarely in practice today)
Modern Relevance	Use of learning cryptography	Historical relevance in war-time ciphers

Real-world Relevance and Evolution

Though both Caesar and Rail Fence ciphers are insecure by modern standards, they are **critical for understanding** how encryption evolved. Modern encryption systems, such as **AES (Advanced Encryption Standard)**, rely on a combination of:

- **Substitution operations (S-Boxes)** – derived from substitution techniques.
- **Permutation (or transposition) steps** – to disperse patterns.

This **confusion and diffusion** approach (Shannon's principles) draws heavily from the concepts behind classical ciphers.

CODE / OUTPUT -

1. Caesar Cipher

```
def caesar_encrypt(text, shift):
```

```
    result = ""
```

```

for char in text:

    if char.isalpha():

        offset = 65 if char.isupper() else 97

        result += chr((ord(char) - offset + shift) % 26 + offset)

    else:

        result += char

return result

def caesar_decrypt(cipher, shift):

    return caesar_encrypt(cipher, -shift)

text = input("Enter text: ")

shift = int(input("Enter shift value (e.g., 3): "))

encrypted = caesar_encrypt(text, shift)

decrypted = caesar_decrypt(encrypted, shift)

print("=" * 45)

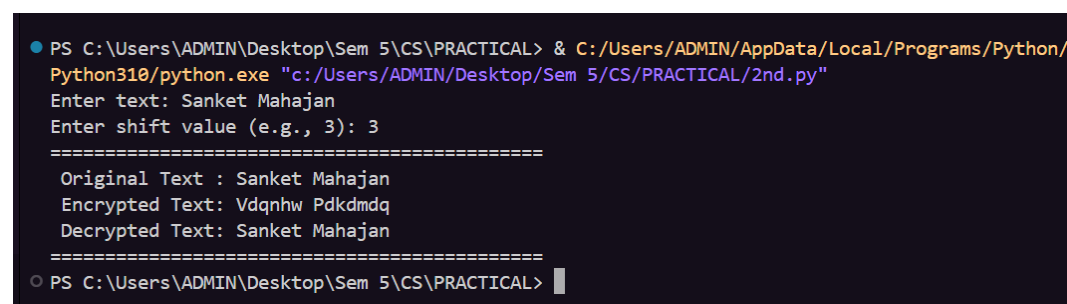
print(f" Original Text : {text}")

print(f" Encrypted Text: {encrypted}")

print(f" Decrypted Text: {decrypted}")

print("=" * 45)

```



```

PS C:\Users\ADMIN\Desktop\Sem 5\CS\PRACTICAL> & C:/Users/ADMIN/AppData/Local/Programs/Python/Python310/python.exe "c:/Users/ADMIN/Desktop/Sem 5/CS/PRACTICAL/2nd.py"
Enter text: Sanket Mahajan
Enter shift value (e.g., 3): 3
=====
Original Text : Sanket Mahajan
Encrypted Text: Vdqnhw Pdkmdq
Decrypted Text: Sanket Mahajan
=====
PS C:\Users\ADMIN\Desktop\Sem 5\CS\PRACTICAL>

```

2. Rail Fence Cipher

```

def encrypt_rail_fence(text, rails):

```

```

fence = [" for _ in range(rails)]

rail = 0

direction = 1

for char in text:

    fence[rail] += char

    rail += direction

    if rail == 0 or rail == rails - 1:

        direction *= -1

return "".join(fence)

def decrypt_rail_fence(cipher, rails):

    length = len(cipher)

    pattern = [[" for _ in range(length)] for _ in range(rails)]

    rail = 0

    direction = 1

    for i in range(length):

        pattern[rail][i] = '*'

        rail += direction

        if rail == 0 or rail == rails - 1:

            direction *= -1

    index = 0

    for r in range(rails):

        for c in range(length):

            if pattern[r][c] == '*' and index < len(cipher):

                pattern[r][c] = cipher[index]

                index += 1

    result = ""

    rail = 0

```

```

direction = 1

for i in range(length):

    result += pattern[rail][i]

    rail += direction

    if rail == 0 or rail == rails - 1:

        direction *= -1

return result

text = input(" Enter the message: ")

rails = int(input(" Enter number of rails (e.g., 3): "))

encrypted = encrypt_rail_fence(text, rails)

decrypted = decrypt_rail_fence(encrypted, rails)

print("=" * 40)

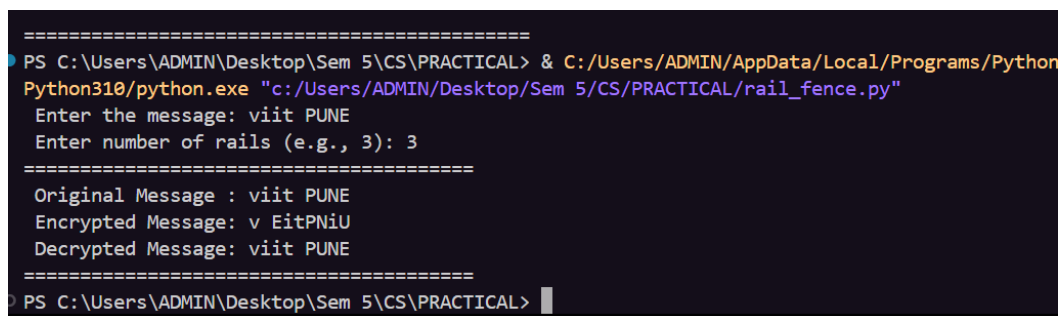
print(f" Original Message : {text}")

print(f" Encrypted Message: {encrypted}")

print(f" Decrypted Message: {decrypted}")

print("=" * 40)

```



```

=====
PS C:\Users\ADMIN\Desktop\Sem 5\CS\PRACTICAL> & C:/Users/ADMIN/AppData/Local/Programs/Python/Python310/python.exe "c:/Users/ADMIN/Desktop/Sem 5/CS/PRACTICAL/rail_fence.py"
Enter the message: viit PUNE
Enter number of rails (e.g., 3): 3
=====
Original Message : viit PUNE
Encrypted Message: v EitPNiU
Decrypted Message: viit PUNE
=====
PS C:\Users\ADMIN\Desktop\Sem 5\CS\PRACTICAL>

```

Conclusion

- Substitution and Transposition are foundational encryption methods that demonstrate how data can be **obscured using simple logic**.
- We implemented two classical algorithms: **Caesar Cipher** (substitution-based) and **Rail Fence Cipher** (transposition-based).
- The Caesar cipher replaces characters, while the Rail Fence rearranges them.
- Both methods have **educational value** and help illustrate **core principles of cryptography**, though they are not secure for practical usage today.

- Understanding these techniques is essential for grasping how **modern cryptographic systems** are structured — where such simple operations are layered, randomized, and combined with complex mathematics to achieve strong encryption.